LeetMastery

System Design Q&A · Print Edition

65 cards · 5 categories

Table of Contents

Design Problems · 29 cards

sd-1 · Design Problems

Design a URL Shortener (TinyURL)

CLARIFY FIRST
- Read-heavy or write-heavy? DAU? Custom aliases?
- Expiry needed? Analytics on clicks?

DATA MODEL
- urls: id, short_code (VARCHAR 7), original_url, user_id, created_at, expires_at
- Short code: Base62 encode a counter (0-9a-zA-Z) → 62^7 = 3.5 trillion URLs

HIGH-LEVEL DESIGN

Client → LB → App Server → Cache (Redis) → DB (MySQL)

KEY DECISIONS
- Code generation: Counter-based (no collisions) > random hash
- Counter: Redis INCR or DB auto-increment with distributed lock
- Redirect: 301 (permanent, browser caches) vs 302 (you track every click)
- Cache: Redis stores short→long, LRU eviction, TTL for expiry

SCALE
- 100M URLs/day = ~1,160 writes/sec, reads 10x higher
- Read replicas for DB, CDN in front of redirect layer

WOW FACTOR
- Analytics: stream click events to Kafka async (never slow down redirects)
- Custom aliases: check uniqueness with DB unique constraint before insert
- Bloom filter: quick "does this alias exist?" check before DB hit

sd-2 · Design Problems

Design Twitter / News Feed

CLARIFY FIRST
- Just feed? Likes/retweets? DAU? Celebrity accounts?
- Real-time or near-real-time? Ranked or chronological?

DATA MODEL
- tweets: id, user_id, content, created_at, media_url
- follows: follower_id, followee_id
- feed_cache: Redis sorted set per user_id → [tweet_ids] scored by timestamp

FAN-OUT STRATEGIES
- Fan-out on WRITE (push): On tweet, write to all followers' feed caches
→ Fast reads, slow writes, bad for celebrities (Lady Gaga = 80M writes)
- Fan-out on READ (pull): Merge followees' timelines at query time
→ Slow reads, storage efficient
- HYBRID (Twitter's actual approach): push for normal users, pull for celebs

FEED SERVICE
- Redis sorted set: ZADD feed:{user_id} timestamp tweet_id
- Pagination: ZREVRANGE feed:{user_id} 0 19

SCALE
- Separate services: Tweet, Feed, User, Media
- Kafka: async fan-out workers consume tweet events

WOW FACTOR
- Hot partition: celebrity tweet causes thundering herd → rate-limit fan-out
- ML ranking: replace pure chronological with engagement-based ranking
- Media: pre-signed S3 upload URL → client uploads direct, bypasses servers

sd-3 · Design Problems

Design a Rate Limiter

CLARIFY FIRST
- Per user? Per IP? Per API key? Distributed (multiple servers)?
- Hard block or soft warn first? Global or per-endpoint limits?

ALGORITHMS

1. Token Bucket: tokens added at fixed rate, consumed per request. Allows bursts.
2. Sliding Window Log: log every request timestamp. Memory heavy.
3. Sliding Window Counter: fixed window + rolling weight. Best balance.
4. Leaky Bucket: queue requests, process at fixed rate. No bursts allowed.

DESIGN

Client → LB → Rate Limiter Middleware → App Server
Middleware checks Redis before forwarding

REDIS IMPLEMENTATION
- Use Lua script: check + decrement is atomic (prevents race conditions)
- Key: rate:{user_id} Value: counter TTL: window size

DISTRIBUTED RATE LIMITING
- Redis cluster: centralized, consistent, small network latency
- Local + Redis: fast local counter first, sync to Redis periodically
- Sloppy counting is acceptable for most use cases

WOW FACTOR
- Return headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset
- Soft limits: warn user at 80% before hard block
- Circuit breaker: if Redis is down, fail open (allow) vs fail closed (deny) — discuss the tradeoff

sd-4 · Design Problems

Design WhatsApp / Chat Application

CLARIFY FIRST
- 1-on-1 only or group chats? Max group size? Message history? Read receipts?
- Offline support? End-to-end encryption?

DATA MODEL
- users: id, phone, name, last_seen
- conversations: id, type (direct/group), name
- conversation_members: conversation_id, user_id, joined_at
- messages: id, conversation_id, sender_id, content, created_at, status

REAL-TIME MESSAGING
- WebSocket: persistent connection per user for real-time push
- Connection Service: maps user_id → server (stored in Redis)
- If recipient on different server → route via message queue (Kafka)

MESSAGE FLOW
1. Sender → WebSocket → Chat Server
2. Chat Server saves message to Cassandra (write-heavy, time-series)
3. Fan-out: publish to Kafka topic per conversation
4. Workers deliver to online recipients via their WebSocket servers
5. Offline users: store in queue, deliver on reconnect with sequence numbers

SCALE
- Cassandra: partition by (conversation_id, month) → fast range reads
- Group chats: cap at 1000 members for fan-out manageability

WOW FACTOR
- E2E encryption: store encrypted blobs, keys never leave client
- Message status: sent → delivered → read (double tick via ACKs back to sender)
- Group chat ≈ Multiplayer card game: same pub/sub pattern underneath

sd-5 · Design Problems

Design Uber / Ride-Sharing

CLARIFY FIRST
- Just matching? Include real-time tracking? Pricing? DAU and # drivers?

DATA MODEL
- drivers: id, current_location (lat/lng), status, vehicle_info
- riders: id, name, payment_method
- trips: id, rider_id, driver_id, status, pickup, dropoff, price, created_at
- Driver location updated every 4 seconds

LOCATION SERVICE (the core problem)
- Redis Geo: GEOADD driver locations, GEODIST, GEORADIUS for nearby search
- Driver heartbeat: update every 4s, TTL=30s (expired entry = offline driver)
- QuadTree or Google S2 for accurate geo-cell partitioning

MATCHING FLOW
1. Rider requests trip → Trip Service creates record (status: SEARCHING)
2. Location Service finds drivers within 5km
3. Matching Service ranks by ETA + rating, sends offer via WebSocket
4. Driver accepts → status: MATCHED → both sides get real-time updates

SCALE
- 1M drivers × 15 updates/min = 250K writes/sec → Redis cluster required
- Surge pricing: supply/demand ratio per geo-cell triggers multiplier

WOW FACTOR
- Trip state machine: SEARCHING→MATCHED→ENROUTE→ARRIVED→INPROGRESS→COMPLETED
- Payment: async after trip ends via Saga pattern (idempotent retries)
- Fraud detection: ML scoring on unusual pickup/dropoff patterns

sd-6 · Design Problems

Design Instagram

CLARIFY FIRST
- Photo upload + feed? Stories? DMs? Start with core photo + feed.
- DAU? Global or regional? Strong consistency on likes needed?

DATA MODEL
- photos: id, user_id, s3_url, caption, created_at
- follows: follower_id, followee_id
- likes: photo_id, user_id, created_at
- feed_cache: Redis sorted set per user_id

PHOTO UPLOAD FLOW
1. Client requests pre-signed S3 URL from Upload Service
2. Client uploads DIRECTLY to S3 (never touches your app servers)
3. S3 event triggers async Transcoding/thumbnail generation
4. Metadata saved to DB, CDN URL stored → photo ready

FEED GENERATION
- Hybrid fan-out: push for normal users, pull for celebrities (>10K followers)
- Denormalize: store enough in feed cache to avoid DB lookups on render

SCALE
- Storage: 1B photos × 3MB avg = 3PB → S3 + Glacier for cold content
- CDN: Akamai/CloudFront serves from edge, near zero origin load for reads

WOW FACTOR
- Stories: separate service, 24hr TTL, ring buffer per user
- Explore feed: ML ranking on engagement signals (not just follows)
- Blurhash: generate colour placeholder on upload → instant perceived performance

sd-7 · Design Problems

Design Google Autocomplete / Search Suggestions

CLARIFY FIRST
- QPS? Personalized results? How fresh must suggestions be? Typo tolerance?
- Top-5 or top-10? Mobile or desktop?

DATA PIPELINE
- Search logs → Kafka stream → Spark aggregation job → frequency DB
- Trie stores prefix → [top 5 queries with frequency scores]
- Updated hourly via offline batch job, not real-time

ARCHITECTURE
- Data Collection: stream every search event to Kafka
- Aggregation: Spark/Flink counts query frequencies with decay factor
- Trie Service: serialized trie in Redis, rebuilt hourly
- API: GET /autocomplete?prefix=face → top 5 in <100ms

STORAGE STRATEGY (prefix table is simpler than trie in practice)
- Store every prefix → top 5 in a DB/cache
- "fa" → ["facebook","fast food","fate"...]
- "fac" → ["facebook","facetime"...]
- Reads are simple key lookups — very fast

SCALE
- Read-heavy: cache results aggressively (TTL 1hr for global suggestions)
- Shard by prefix first character → 26 shards easily handles load

WOW FACTOR
- Personalization: blend global top-K with user's personal search history (weighted)
- Decay factor: recent searches score higher than old ones
- Typo tolerance: BK-tree or edit distance for fuzzy prefix matching

sd-8 · Design Problems

Design YouTube Video Upload & Streaming

CLARIFY FIRST
- Just upload pipeline? Include streaming/delivery? Comments/likes?
- Video lengths? Global or regional? Live streaming?

DATA MODEL
- videos: id, user_id, title, status, s3_key, created_at
- video_chunks: video_id, chunk_index, s3_url
- video_formats: video_id, resolution, s3_url, duration

UPLOAD PIPELINE
1. Client splits video into 5MB chunks
2. Upload Service returns pre-signed S3 URLs per chunk
3. Client uploads chunks to S3 IN PARALLEL
4. Client signals completion → Transcoding Job queued in Kafka
5. Workers transcode to 1080p/720p/480p/360p via FFmpeg
6. All versions stored in S3, metadata DB updated → status: READY

STREAMING (Adaptive Bitrate)
- HLS/DASH: client requests .m3u8 playlist → downloads 10-sec segments
- Client switches quality automatically based on bandwidth measurement
- CDN serves all segments from edge — origin rarely hit

SCALE
- 500 hours of video uploaded per minute → async transcoding queue critical
- CDN absorbs 90%+ of traffic; origin only for cache misses

WOW FACTOR
- Resume upload: track uploaded chunks in Redis by session → resume on reconnect
- Content moderation: ML scan transcoded video async, don't block upload flow
- Thumbnail: auto-generate keyframes + allow custom upload, store in S3

sd-9 · Design Problems

Design a Payment Processing System

CLARIFY FIRST
- Card payments only? Refunds? International/multi-currency? Retry logic?
- Most critical requirement: exactly-once processing (NO double charges)

DATA MODEL
- payments: id, user_id, amount, currency, status, idempotency_key, created_at
- payment_events: payment_id, event_type, timestamp, metadata (audit log)

ARCHITECTURE
Client → Payment API → Payment Service → Payment Gateway (Stripe/Braintree)
Async confirmation via webhook → update payment status

IDEMPOTENCY (the key insight)
- Client generates UUID idempotency_key for every request
- Before processing: check if key exists in DB → return same response if yes
- Prevents duplicate charges on network timeout + retry

DISTRIBUTED TRANSACTIONS — SAGA PATTERN
Order created → Payment charged → Inventory reserved → Notification sent
If payment fails → compensating transactions run in reverse (cancel order, release inventory)

WOW FACTOR
- At-least-once delivery + idempotent consumers = effectively exactly-once
- Reconciliation: nightly batch job compares your DB vs gateway records (catch any drift)
- PCI compliance: never store raw card numbers — use tokenization (Stripe handles this)
- Outbox pattern: write payment event to DB and message queue atomically

sd-10 · Design Problems

Design a Notification System

CLARIFY FIRST
- Which channels? Push (mobile), email, SMS, in-app?
- Priority levels? Do-not-disturb preferences? Delivery guarantees?

DATA MODEL
- notifications: id, user_id, type, title, body, status, created_at
- user_preferences: user_id, channel, enabled, quiet_hours
- notification_logs: notification_id, channel, status, sent_at

ARCHITECTURE
Event Source → Message Queue (Kafka) → Fan-out Workers → Channel Gateways
- Push: FCM (Android) / APNs (iOS)
- Email: SendGrid / SES
- SMS: Twilio

FLOW
1. Service emits event (OrderShipped, FriendRequest, etc.) to Kafka
2. Notification Service consumes event, checks user preferences
3. Creates notification records, fans out to channel-specific queues
4. Channel workers pick up jobs, call respective gateways
5. Update delivery status via webhook callbacks

SCALE
- Decouple fan-out: don't block the source service, async always
- Priority queues: critical alerts (payment failed) go to high-priority queue

WOW FACTOR
- Template service: centralize notification copy, A/B testable
- Rate limiting: max 3 push notifications/hour per user (avoid spam)
- Idempotency: deduplicate events with event_id to avoid double-sending
- Analytics: track open rates, click rates per notification type

sd-11 · Design Problems

Design a Distributed Key-Value Store (like DynamoDB)

CLARIFY FIRST
- Strong or eventual consistency? Read-heavy or write-heavy?
- How much data? Single-region or multi-region?

CORE ARCHITECTURE
- Consistent Hashing: nodes on ring, key hashes to position, routes to next node
- Virtual nodes: each physical node = 150 ring positions → even distribution
- Replication: store N copies on N consecutive clockwise nodes (N=3 typical)

QUORUM READS/WRITES
- W = minimum nodes that must ack a write (typically 2)
- R = minimum nodes that must respond to a read (typically 2)
- W + R > N = strong consistency. W + R ≤ N = eventual consistency

CONFLICT RESOLUTION
- Vector clocks: each write tagged with version vector
- Last-write-wins: use timestamp (simpler, can lose concurrent writes)
- CRDTs: data types that merge automatically (counters, sets)

FAILURE HANDLING
- Hinted handoff: if target node down, store hint on another, deliver on recovery
- Anti-entropy: Merkle trees detect diverged replicas and sync differences
- Gossip protocol: nodes share cluster membership — no central coordinator

WOW FACTOR
- CAP choice: Dynamo chooses AP (always write, reconcile later) = Cassandra
- Sloppy quorum: accept writes to ANY W healthy nodes, not just "correct" ones
- Bloom filters: check if key might exist before expensive disk read

sd-12 · Design Problems

Design Dropbox / File Sync System

CLARIFY FIRST
- Sync across devices? Sharing/collaboration? Conflict resolution?
- File size limits? Versioning / undo?

DATA MODEL
- files: id, user_id, name, size, content_hash, version, s3_key, updated_at
- file_chunks: file_id, chunk_index, chunk_hash, s3_key
- file_events: file_id, event_type, timestamp (for sync log)

UPLOAD FLOW
1. Client watches filesystem for changes (inotify/FSEvents)
2. On change: split file into 4MB chunks, hash each chunk
3. Check with server which chunks are new (delta sync)
4. Upload ONLY new/changed chunks to S3
5. Update file metadata in DB

SYNC ACROSS DEVICES
- Long-poll or WebSocket: server pushes file_events to connected clients
- Each client maintains local sync cursor (last_event_id)
- On reconnect: pull all events since cursor → apply changes

DEDUPLICATION
- Content-addressable storage: chunk_hash as S3 key
- If another user uploads same file → chunk already exists, no re-upload needed
- Saves 30-40% storage across a large user base

WOW FACTOR
- Conflict handling: if two devices edit same file → create conflict copy (like Dropbox does)
- Bandwidth: delta sync (only changed chunks) crucial for large files
- Resumable: track uploaded chunks per session in Redis → resume on disconnect

sd-23 · Design Problems

Design a Web Crawler

CLARIFY FIRST
- Scope: entire web or specific domain? Frequency of re-crawl? How many pages?
- Store HTML? Extract links only? Handle JavaScript-rendered pages?

DATA MODEL
- urls_to_crawl: id, url, priority, depth, added_at (the frontier queue)
- crawled_pages: id, url, content_hash, crawled_at, status_code, links_found
- Dedup store: URL hash → crawled flag (Bloom filter or Redis SET)

HIGH-LEVEL DESIGN
URL Frontier → Fetcher Workers → HTML Parser → Link Extractor → URL Filter → Frontier
↓
Content Storage (S3)

KEY COMPONENTS
- URL Frontier: priority queue (BFS = breadth-first). Partitioned by domain.
- Politeness: respect robots.txt. 1 request/domain/second. Delay per host.
- DNS resolver: cache DNS lookups (expensive). Batch resolution.
- Content dedup: hash page content → skip if already seen (SimHash for near-dups)

SCALE
- 1B pages, 500 bytes avg = 500GB/day → distribute frontier across machines
- Multi-threaded fetchers — one thread per domain (politeness)
- Bloom filter: 1B URLs, 1% false positive rate = ~1.2GB RAM (vs 20GB for hash set)

WOW FACTOR
- SimHash: detect near-duplicate pages (same article, different ads) — not just exact dups
- Recrawl priority: news sites = every hour; static docs = every month
- URL trap detection: infinite pagination loops → max depth limit per domain

sd-24 · Design Problems

Design a Real-time Leaderboard

CLARIFY FIRST
- Global leaderboard or per-game? Real-time or near-real-time? Top-100 or full rank?
- Score updates frequency? Billions of users or millions?

DATA MODEL
- scores: user_id, score, updated_at (source of truth in DB)
- leaderboard cache: Redis sorted set → ZADD leaderboard score user_id

REAL-TIME LEADERBOARD (Redis Sorted Set)
- ZADD leaderboard 9500 user:123 → add/update score
- ZREVRANK leaderboard user:123 → get rank (0-indexed)
- ZREVRANGE leaderboard 0 99 WITHSCORES → top 100 with scores
- All operations O(log N) — handles millions of users easily

FLOW
1. User completes game → score event → Kafka topic
2. Score Service consumes event → updates Redis sorted set atomically
3. Score Service also writes to DB (async) for durability
4. Leaderboard API reads from Redis — sub-millisecond response

SCALE
- Redis sorted set handles 100M users comfortably on a single instance
- Shard by game_id if running thousands of separate leaderboards
- Historical leaderboards: store snapshots in DB (daily/weekly), don't keep in Redis

WOW FACTOR
- Friend leaderboard: ZRANGEBYSCORE filtered by friend list (store friend IDs in Redis SET)
- Ties: ZADD uses float scores → append microsecond timestamp as decimal for tiebreak
- Near-real-time: batch score updates every 1s with Lua script for atomic multi-update

sd-25 · Design Problems

Design Google Calendar / Scheduling System

CLARIFY FIRST
- Personal calendar only or shared? Recurring events? Meeting room booking? Timezone handling?
- Conflict detection needed? Notifications? How many users?

DATA MODEL
- events: id, creator_id, title, start_time (UTC), end_time (UTC), recurrence_rule, location
- event_attendees: event_id, user_id, status (accepted/declined/pending)
- calendar_shares: calendar_id, shared_with_user_id, permission
- notifications: event_id, user_id, notify_at, channel

HIGH-LEVEL DESIGN
Client → API Gateway → Event Service → DB (PostgreSQL)
↓
Notification Scheduler → Queue → Workers → FCM/Email

KEY CHALLENGES
Recurring Events:
- Store recurrence rule (RFC 5545: RRULE=FREQ=WEEKLY;BYDAY=MO,WE,FR)
- Expand on-the-fly for display (don't store each occurrence)
- Exception: if one occurrence edited → store as override record

Timezone Handling:
- Always store in UTC. Display in user's local timezone.
- Use IANA timezone IDs (America/New_York), not offsets (DST changes)

Conflict Detection:
- SELECT * WHERE start < newEnd AND end > newStart AND room_id = ?
- Index on (room_id, start_time, end_time) for fast overlap queries

SCALE
- Notification scheduler: cron job finds events in next 15 mins → queue jobs
- Fan-out: large meeting (1000 attendees) → async worker fan-out

WOW FACTOR
- Pub/sub for live updates: attendee accepts → all viewers see update instantly via WebSocket
- Smart scheduling: suggest times when all attendees are free (INTERSECT availability windows)

sd-26 · Design Problems

Design a Real-time Collaborative Document Editor (Google Docs)

CLARIFY FIRST
- How many concurrent editors per doc? Conflict resolution strategy?
- Rich text or plain text? Version history? Offline support?

DATA MODEL
- documents: id, title, owner_id, created_at
- document_versions: doc_id, version, snapshot_content, created_at
- operations: id, doc_id, user_id, op_type, position, content, version, timestamp

REAL-TIME SYNC
- WebSocket: each user holds a persistent connection to a Doc Server
- Operations sent as delta changes (not full document) — efficient

CONFLICT RESOLUTION
Operational Transformation (OT) — Google Docs approach:
- Transform concurrent operations against each other so all clients converge
- Insert at pos 5 by User A + Delete at pos 3 by User B → transform A's op after B's
- Complex to implement correctly but proven at scale

CRDTs (Conflict-Free Replicated Data Types) — Figma/Notion approach:
- Data structures that automatically merge without coordination
- No central server needed for conflict resolution
- Easier to reason about, growing adoption

ARCHITECTURE
Client → WebSocket → Doc Server → Operation Queue (Redis) → DB
↓
Broadcast to other connected clients

WOW FACTOR
- Cursor presence: show each user's cursor position to others (low-priority WebSocket channel)
- Snapshot + ops: store full snapshot every 100 ops → reconstruct by replaying ops from snapshot
- Offline: queue ops locally, send on reconnect, server reconciles with OT/CRDT

sd-27 · Design Problems

Design a Parking Lot System

CLARIFY FIRST
- Multiple floors? Vehicle types (compact, large, motorcycle)? Reservations or drive-up only?
- Pricing: flat rate or hourly? Payment at kiosk or exit gate?

DATA MODEL
- parking_lots: id, name, address, total_spots
- parking_spots: id, lot_id, floor, spot_number, type, status (FREE/OCCUPIED/RESERVED)
- tickets: id, spot_id, vehicle_id, entry_time, exit_time, amount_charged
- reservations: id, spot_id, user_id, start_time, end_time, status

CORE FLOW
Entry: Scan plate → Find available spot (matching vehicle type) → Assign → Print ticket → Open gate
Exit: Scan ticket → Calculate fee → Process payment → Update spot → Open gate

SPOT ASSIGNMENT
- Strategy pattern: NearestEntry (for accessibility), LowestFloor (default), Random
- In-memory count per type per floor → fast availability check
- DB update on actual assignment (with optimistic locking to prevent double-assign)

KEY CONSIDERATIONS
- Concurrency: two cars assigned same spot → use DB transaction + unique constraint
- Spot types: COMPACT ← can also fit MOTORCYCLE. LARGE → can park any vehicle.
- Pricing: strategy pattern → HourlyPricing, FlatRatePricing, WeekendPricing

SCALE
- Single lot: simple DB, no distributed complexity needed
- Multi-city network: Central Service per region, aggregate spot counts

WOW FACTOR
- License plate recognition (LPR): camera at entry reads plate → auto-lookup reservation
- Dynamic pricing: surge on busy periods (airports, events) — adjust base rate per time window
- Electric vehicle spots: track charger availability separately, longer minimum dwell time

sd-28 · Design Problems

Design Google PageRank

CLARIFY FIRST
- Full web-scale or subset? Real-time updates or batch? Combined with content ranking?

CORE ALGORITHM
PageRank = (1-d) + d × Σ(PageRank(linking_page) / OutLinks(linking_page))
- d = damping factor (0.85) — probability of following a link vs random jump
- Iterates until values converge (typically 50-100 iterations)
- Intuition: a page is important if important pages link to it

DISTRIBUTED COMPUTATION
- Web graph: nodes=pages, edges=hyperlinks → too large for one machine (4T+ pages)
- MapReduce / Spark: distribute adjacency matrix across cluster
- Map: emit (destination_url, rank_contribution)
- Reduce: sum contributions per URL, apply damping factor

DATA MODEL
- pages: id, url, current_rank, out_degree
- links: source_id, destination_id (sparse adjacency matrix)
- Store as CSR (Compressed Sparse Row) format — memory efficient

SCALE
- Sparse matrix: only store non-zero edges (most pages link to few others)
- Partition graph: split by URL hash across workers
- Convergence check: stop when max rank change < 0.001

WOW FACTOR
- Dangling nodes: pages with no outbound links → redistribute their rank uniformly
- Topic-sensitive PageRank: separate rank vectors per topic category
- Modern search: PageRank is one of 200+ signals — content quality, freshness, clicks also matter

sd-29 · Design Problems

Design Memcache / Redis (LRU Cache)

CLARIFY FIRST
- Single node or distributed? Max memory? Eviction policy? Persistence needed?
- Read/write ratio? Key-value only or richer data structures?

DATA STRUCTURES
- Hash table: O(1) get/set by key
- Doubly linked list: maintain LRU order (head = most recent, tail = least recent)
- Combined: HashMap<key, ListNode> + DoublyLinkedList

LRU OPERATIONS
- GET(key): find in map → move node to head → return value. O(1).
- PUT(key, val): exists → update + move to head. New → add at head.

If at capacity → evict tail node → remove from map. O(1).

DISTRIBUTED CACHE
- Consistent hashing: distribute keys across N cache nodes
- Client-side sharding: client hashes key → picks cache node directly (no proxy)
- Replication: each cache node has a hot standby for HA

SCALE
- Memcached: multi-threaded, pure LRU, key-value only, no persistence
- Redis: single-threaded (fast), rich data types, optional persistence (RDB/AOF)
- Cluster mode: Redis Cluster uses 16,384 hash slots assigned to nodes

WOW FACTOR
- Thundering herd on cold start: warm cache gradually (preload hot keys on startup)
- Cache stampede: add jitter to TTL (not all keys expire simultaneously)
- Write strategies: cache-aside vs write-through vs write-behind — discuss tradeoffs per use case
- Redis persistence: RDB (snapshots) for fast restart, AOF (append-only log) for durability

sd-30 · Design Problems

Design Ads Fraud Detection System

CLARIFY FIRST
- What types of fraud? Click fraud? Impression fraud? Account fraud?
- Latency requirement? Block in real-time (<100ms) or flag for review?
- How many events/sec?

DATA MODEL
- click_events: id, ad_id, user_id, ip, device_fingerprint, timestamp, geo
- fraud_scores: event_id, score, features, model_version, flagged_at
- blocked_entities: type (ip/user/device), value, reason, blocked_until

ARCHITECTURE

Ad Click → Kafka → Stream Processor (Flink/Spark Streaming) → Fraud Scorer → Decision
↓
Redis (real-time state)

DETECTION SIGNALS
- Click velocity: >50 clicks/IP/hour → suspicious
- CTR anomaly: CTR 10x above baseline for this ad → bot traffic
- Device fingerprint: same fingerprint, 100 different IPs → fraud ring
- Geographic: click from country where ad doesn't run → invalid traffic
- Time pattern: clicks at exactly 1-second intervals → bot

REAL-TIME SCORING
- Feature extraction in stream: sliding window counts in Redis (INCR + EXPIRE)
- ML model inference: lightweight gradient boosting model (<5ms latency)
- Rule engine: hard rules evaluated first (fast), ML score as second layer

WOW FACTOR
- Graph analysis: detect fraud rings — clusters of IPs/devices/users that co-occur suspiciously
- Feedback loop: confirmed fraud → retrain model weekly with new labels
- Advertiser dashboard: show invalid traffic % per campaign with breakdown

sd-46 · Design Problems

Design Spotify / Music Streaming Service

CLARIFY FIRST
- Audio streaming only or podcasts/videos too? (assume audio + podcasts)
- Free vs premium tiers? Offline playback? Social features?
- Scale: 100M users, 80M songs, 30M DAU

DATA MODEL

Song: song_id, title, artist_id, duration, s3_url, bitrate_versions[]
User: user_id, plan, liked_songs[], playlists[]
Play event: user_id, song_id, timestamp, duration_played, device

HIGH-LEVEL DESIGN

Client → CDN (audio delivery) → API Gateway → Services:
- Catalog Service: song metadata, search (Elasticsearch)
- Streaming Service: signed S3 URLs, adaptive bitrate
- Recommendation Service: ML model (collaborative filtering)
- Playlist Service: CRUD with Redis cache
- Play History: Kafka → analytics pipeline → DynamoDB

KEY DECISIONS

Audio storage: multiple bitrates (128kbps, 256kbps, 320kbps) on S3
Adaptive streaming: HLS chunks (10s segments), client adjusts quality
CDN pre-warm: popular songs at edge nodes globally
Offline: encrypted DRM files stored locally, license server validates

SCALE

Peak streaming: pre-signed S3 URLs bypass app servers
Recommendation: batch ML nightly + real-time feedback loop
Song upload: background transcode to multiple bitrates via worker queue

WOW FACTOR
- Gapless playback: pre-buffer next track while current track plays
- "Audio graph" — Spotify's recommendation is a graph of 30M songs connected by listener co-occurrence
- Podcast vs music separated infra — different CDN and latency SLAs

sd-47 · Design Problems

Design Netflix / Video Streaming Service

CLARIFY FIRST
- Upload (content team) vs streaming (users) — both sides
- Global scale: 200M subscribers, 15% of global internet traffic
- HD/4K streaming, multiple devices simultaneously

DATA MODEL

Video: video_id, title, s3_keys[], resolutions[], duration, subtitles[]
User: user_id, subscription, watchlist[], viewing_history[]
ViewingSession: session_id, user_id, video_id, resume_position, quality

HIGH-LEVEL DESIGN

Upload pipeline: raw video → encoding farm → multiple resolutions (360p/720p/1080p/4K) → stored S3 → CDN
Streaming pipeline: user → Open Connect CDN (Netflix's private CDN) → adaptive bitrate streaming
Services:
- Content Catalog: metadata, search (Elasticsearch)
- Recommendation: ML service (Netflix uses ~80% of views are recommended)
- Streaming: signed manifest URLs, adaptive HLS/DASH
- Billing: Stripe integration
- Analytics: Kafka → Spark → Redshift

KEY DECISIONS

Netflix Open Connect: private CDN in ISP data centers — reduces transit costs 80%
Adaptive bitrate: DASH protocol, client monitors bandwidth every 2s and adjusts
Encoding: VP9/H.265 for bandwidth efficiency, parallel encoding farm
DRM: Widevine (Android/Chrome), FairPlay (Apple), PlayReady (Microsoft)

SCALE

Pre-position popular content at edge before prime time (predictive caching)
Database: Cassandra for viewing history (time-series write heavy)
Chaos Engineering: Netflix Simian Army intentionally kills servers to test resilience

WOW FACTOR
- Studio-quality compression: Netflix's encoding cuts file size 20% vs standard H.264
- Personalized thumbnails: same movie shows different thumbnails per user based on taste
- 5 nines availability via multi-region active-active with Zuul API gateway

sd-48 · Design Problems

Design Tinder / Dating App

CLARIFY FIRST
- Core feature: swipe left/right, matches, messaging
- Scale: 50M users, 1.6B swipes/day, 26M matches/day
- Location-based matching radius?

DATA MODEL

User: user_id, profile{name, bio, photos[], age, gender, location}
Swipe: swiper_id, swipee_id, direction (left/right), timestamp
Match: match_id, user1_id, user2_id, created_at
Message: match_id, sender_id, content, timestamp

HIGH-LEVEL DESIGN

Services:
- Profile Service: CRUD + photo upload (S3 + CDN)
- Discovery Service: candidate pool generation + ranking
- Swipe Service: write swipes, detect mutual likes → create match
- Match Service: notify both users via WebSocket/push
- Chat Service: WebSocket per match, messages in Cassandra

KEY DECISIONS

Discovery algorithm:
1. Location filter: PostGIS / Geohash to find users within radius
2. Preference filter: age, gender preferences
3. Already-swiped filter: Redis bloom filter (seen set per user)
4. Rank candidates: score by ELO-like attractiveness rating + recency

Swipe storage: Redis sorted set per user for fast bloom filter check
Match detection: when user A swipes right on B, check if B already swiped right on A → match!
Photos: S3 + CloudFront CDN, multiple sizes (thumbnail, full)

SCALE

Swipe volume: write to Redis first, async flush to Cassandra
Recommendation batch: pre-compute candidate stacks nightly, refresh on location change
Location updates: only update when user moves > 100m (save writes)

WOW FACTOR
- "Smart Photos" — A/B tests your photos automatically and surfaces best-performing one
- ELO score: similar to chess rating — your profile is shown more to similar-scored users
- Geohash partitions the world into cells, swipe data sharded by geohash region

sd-49 · Design Problems

Design Reddit / Discussion Platform

CLARIFY FIRST
- Core: posts, comments (threaded), upvotes, subreddits
- Scale: 50M DAU, 100K+ communities, 18+ votes/day
- Real-time vote counts? Hot/New/Top ranking?

DATA MODEL

Post: post_id, subreddit_id, user_id, title, body/url, score, created_at
Comment: comment_id, post_id, parent_comment_id, user_id, body, score
Vote: user_id, entity_id, entity_type, direction (+1/-1)
Subreddit: sub_id, name, rules, moderators[], subscriber_count

HIGH-LEVEL DESIGN

Services:
- Post/Comment Service: CRUD, stored in PostgreSQL
- Vote Service: high write volume → Redis counter + async DB sync
- Feed/Ranking Service: computes Hot/New/Top scores
- Search: Elasticsearch for cross-subreddit search
- Notification Service: mentions, replies → Kafka → push/email

Ranking algorithms:
- Hot: score / (age + decay_constant)^gravity — Reddit's actual formula
- Top: pure vote score
- New: chronological

KEY DECISIONS

Vote counting: Redis INCR for real-time, periodic flush to Postgres
Threaded comments: nested set model or adjacency list (parent_id) — adjacency simpler, nested faster for reads
Feed: each subreddit feed is precomputed and cached in Redis
CDN: images/videos in S3 + CloudFront

SCALE

Read-heavy: aggressive Redis caching for hot posts/comments
Vote fraud: rate limiting + IP analysis to prevent brigading
Fanout: when r/worldnews post goes viral, fan out to 30M subscriber feeds selectively

WOW FACTOR
- Reddit's Hot algorithm is public — "gravity" constant (1.8) controls how fast posts decay
- Awards system: premium awards store in separate service, easy to add without core changes
- Mod tools: separate moderation service with different SLA — doesn't block main read path

sd-50 · Design Problems

Design Food Delivery (DoorDash / Uber Eats)

CLARIFY FIRST
- Full flow: customer orders → restaurant accepts → dasher picks up → delivers
- Real-time location tracking, ETA updates
- Scale: 25M orders/day, 5M dashers

DATA MODEL

Order: order_id, customer_id, restaurant_id, dasher_id, items[], status, total
Restaurant: restaurant_id, menu[], location, operating_hours, avg_prep_time
Dasher: dasher_id, current_location, status (available/busy), rating
Location: dasher_id, lat, lng, timestamp (write-heavy, time-series)

HIGH-LEVEL DESIGN

Services:
- Order Service: order lifecycle state machine (placed→confirmed→ready→picked→delivered)
- Restaurant Service: menu, availability, ETA estimation
- Dasher Matching Service: geospatial query for nearby available dashers
- Location Service: dasher GPS updates (every 5s) → Redis + time-series DB
- Notification: real-time updates via WebSocket/push at each state change
- Payment: Stripe + fraud detection

KEY DECISIONS

Dasher matching: Geohash cells, find dashers within radius, rank by distance + load
ETA: restaurant prep time + pickup travel time + delivery travel time (ML model)
Location updates: Redis geospatial (GEOADD) for O(log N) nearest dasher queries
Order state machine: Kafka events for each transition, audit log in DynamoDB

SCALE

Location writes: 5M dashers × 1 update/5s = 1M writes/sec → Redis + async flush
Peak times (dinner rush): autoscale dasher matching service
Restaurant surge: dynamic pricing and longer ETAs surfaced proactively

WOW FACTOR

- Predicted demand: ML model pre-positions dashers near restaurants before dinner rush
- "Project DASH" — DoorDash pre-computed delivery zones geofenced by Geohash to minimize dispatch time
- Multi-order batching: one dasher carries 2 orders to nearby locations — 40% efficiency gain

sd-51 · Design Problems

Design Amazon E-Commerce / Product Catalog

CLARIFY FIRST

- Scope: product catalog, search, cart, checkout, order tracking
- Scale: 300M products, 200M customers, Black Friday peaks 20x normal
- Inventory accuracy required? Third-party sellers?

DATA MODEL

Product: product_id, title, description, images[], price, inventory_count, seller_id
Order: order_id, user_id, items[], status, payment_id, shipping_address
Cart: user_id → [product_id, quantity, price_snapshot]
Inventory: product_id, warehouse_id, quantity, reserved_quantity

HIGH-LEVEL DESIGN

Services:
- Catalog Service: product data in DynamoDB (read-heavy), Elasticsearch for search
- Search: Elasticsearch with typo tolerance, filters, facets, personalized ranking
- Cart Service: Redis (session storage), checkout copies to Order Service
- Order Service: state machine (placed→payment→fulfillment→shipped→delivered)
- Inventory Service: pessimistic locking on reserve, saga pattern for rollback
- Recommendation: collaborative filtering ("customers also bought")
- Payment: idempotent Stripe integration

KEY DECISIONS

Inventory consistency: optimistic locking for catalog reads, pessimistic reserve at checkout
Cart: Redis for fast reads, persisted to DB on checkout
Search ranking: ML-based, considers click-through rate, conversion rate, relevance
Flash sales: queue-based checkout to prevent oversell, Redis atomic DECR for stock

SCALE

Read/write split: read replicas for catalog, primary only for inventory writes
CDN: product images on S3 + CloudFront, compressed and resized per device
Black Friday: autoscaling + pre-scaled capacity, circuit breakers on downstream calls

WOW FACTOR

- Amazon's product page = 150+ microservices stitched together via server-side composition
- Inventory "reservation" vs "commit" — stock reserved at add-to-cart, committed at payment
- A/B testing at massive scale: 1000+ experiments running simultaneously

HIGH-LEVEL DESIGN

Services:
- Listing Service: CRUD, stored in PostgreSQL + Elasticsearch for search
- Search Service: geo-search (PostGIS), date availability filter, price filter, ranking
- Booking Service: availability check → reserve → payment → confirm (saga pattern)
- Availability Service: calendar management, conflict detection
- Review Service: post-stay reviews, aggregate ratings
- Payment: Stripe + escrow (holds funds until 24h after check-in)
- Notification: booking confirmations, check-in reminders

KEY DECISIONS

Double-booking prevention: pessimistic lock on availability rows during booking transaction
Search: Elasticsearch for text + PostGIS for geo, date filter as post-query step
Pricing: dynamic pricing engine considers local events, seasonality, demand
Availability calendar: separate table per listing × date — fast O(1) lookup

SCALE

Search: read replicas + Elasticsearch cluster; most traffic is search not booking
Booking: strong consistency required → PostgreSQL with row-level locking
Media: photos stored S3 + CloudFront CDN, watermarked thumbnails generated on upload

WOW FACTOR

- Fraud detection: ML model scores every booking — unusual location combos flagged
- "Instant book" vs "request to book" — different code paths, different SLAs
- Airbnb's pricing ML considers 70+ signals including local hotel rates and flight searches

sd-52 · Design Problems

Design Airbnb / Property Booking Platform

CLARIFY FIRST

- Host lists property → guest searches → books → stays
- Scale: 7M listings, 150M users, 2M+ stays/night
- Real-time availability? Double-booking prevention critical

DATA MODEL

Listing: listing_id, host_id, title, location, price_per_night, amenities[], photos[]
Booking: booking_id, listing_id, guest_id, check_in, check_out, status, total_price
Availability: listing_id, date, status (available/booked/blocked)
Review: listing_id, booking_id, guest_id, rating, text

HIGH-LEVEL DESIGN

Map tiles: pre-rendered vector tiles stored in S3 by zoom/x/y, served via CDN
Services:
- Tile Service: client requests tiles by zoom level + viewport → CDN → S3
- Search Service: place search (Elasticsearch geo), autocomplete
- Routing Service: Dijkstra / A* on road graph, real-time traffic weights
- ETA Service: ML model using current traffic + historical patterns + route
- Traffic Ingestion: GPS pings from 1B devices → Kafka → aggregate speed per road segment

KEY DECISIONS

Map tiles: vector tiles (small, scalable) over raster; client renders locally
Road graph: compressed adjacency list, bidirectional A* for fast routing
Traffic: anonymous GPS from Google Maps users aggregated per road segment every 2min
Routing: precomputed highway routes cached, recalculated for local roads with traffic

SCALE

Tile CDN: 99% of tile requests served from CDN — huge traffic at zoom-in events (concerts etc)
Search: Elasticsearch geo-query + autocomplete prefix tree
Traffic processing: Kafka consumer groups aggregate GPS → Redis per road segment

WOW FACTOR

- Google's road graph has 30M+ road segments globally — stored as compressed adjacency list
- Contraction Hierarchies: preprocessing speeds up Dijkstra 1000x by pre-computing "highway" shortcuts
- Live traffic = 1B GPS pings anonymized, aggregated, smoothed per 100m road segment

sd-54 · Design Problems

Design Zoom / Video Conferencing System

CLARIFY FIRST

- Core: 1:1 calls, group meetings (up to 1000 participants), screen share, recording
- Scale: 300M daily meeting participants, peak COVID: 3B meeting minutes/day
- Low latency critical (< 150ms end-to-end)

DATA MODEL

Meeting: meeting_id, host_id, participants[], start_time, recording_url, status
Participant: user_id, meeting_id, join_time, audio_track, video_track, connection_quality
Recording: meeting_id, s3_url, duration, transcript

HIGH-LEVEL DESIGN

Signaling Server: WebSocket server for call setup (SDP offer/answer exchange)
Media Servers: handle audio/video mixing and relay
- Small meetings (< 5): P2P WebRTC (no media server)
- Large meetings: SFU (Selective Forwarding Unit) — relay streams without mixing
- Very large: MCU (Multipoint Control Unit) — mix into composite stream

Services:
- Meeting Service: create/join/leave meetings, persistent state in Redis + PostgreSQL
- Auth Service: meeting passwords, waiting room, lobby
- Recording Service: capture streams → S3, async transcription
- Quality Service: adaptive bitrate based on participant bandwidth

KEY DECISIONS

P2P vs SFU vs MCU:
- P2P: best quality, no server cost, but $O(N^2)$ bandwidth for N participants
- SFU: each client sends 1 stream up, receives N-1 streams — scales to ~50 participants
- MCU: server mixes all streams → 1 stream per client — CPU heavy but bandwidth efficient

Jitter buffer: absorbs network packet jitter, smooth playback
Forward Error Correction (FEC): sends redundant packets so loss doesn't freeze video

SCALE

Media servers: geo-distributed, participant routed to nearest region
Bandwidth: participants send at adaptive bitrate (720p → 360p → audio-only as network degrades)
Recording: async — store raw streams in S3, post-process after meeting ends

WOW FACTOR

- Zoom's "subspace tunneling" — proprietary protocol layered over UDP for better packet recovery than standard WebRTC
- Virtual backgrounds: ML model runs locally (client-side) — no video sent to Zoom servers
- Active speaker detection: audio VAD (voice activity detection) to switch camera focus automatically

Core Concepts · 20 cards

sd-13 · Core Concepts

Explain the CAP Theorem. How do you apply it when choosing a database?

CAP THEOREM

In a distributed system you can only guarantee 2 of 3:
- C — Consistency: all nodes see the same data at the same time
- A — Availability: every request gets a response (not necessarily latest)
- P — Partition Tolerance: system works despite network failures

THE REALITY

Network partitions ALWAYS happen in distributed systems.
So the real choice is: CP or AP — P is never optional.

CP SYSTEMS (sacrifice availability)
- HBase, Zookeeper, etcd, MongoDB (default)
- Rejects requests during partition to maintain consistency
- Use when: banking, inventory (wrong answer = serious problem)

AP SYSTEMS (sacrifice consistency)
- Cassandra, DynamoDB, CouchDB
- Returns potentially stale data — eventually consistent
- Use when: social likes, recommendations, search (stale = acceptable)

WOW FACTOR

- PACELC extends CAP: even WITHOUT partitions you choose Latency vs Consistency
- "Eventual consistency" = all replicas WILL agree, just not instantly
- Cassandra lets you tune per-query: ONE=AP, QUORUM=balanced, ALL=CP
- Real example: bank transfer = CP needed. Twitter like count = AP is fine

sd-14 · Core Concepts

What are the main caching strategies? When do you use each?

CACHE-ASIDE (Lazy Loading) — Most Common
- App checks cache first → miss → read DB → write to cache
- Pro: only cache what's needed, resilient to cache failure
- Con: 3 trips on miss, cold start problem
- Use: general purpose, sessions, API responses

WRITE-THROUGH
- Write to cache AND DB simultaneously on every write
- Pro: cache always fresh, no read miss after write
- Con: higher write latency, caches data that may never be read
- Use: when reads closely follow writes (profile updates)

WRITE-BEHIND (Write-Back)
- Write to cache immediately, flush to DB asynchronously
- Pro: extremely fast writes
- Con: data loss if cache crashes before flush
- Use: high-write workloads where small loss is acceptable

EVICTION POLICIES
- LRU (Least Recently Used): evict oldest untouched → Redis default
- LFU (Least Frequently Used): evict least popular over time
- TTL: always set expiry to prevent stale data buildup

WOW FACTOR
- Thundering herd: cache key expires → 10K requests all miss simultaneously
- Fix: mutex lock on first miss, or probabilistic early recompute (PER algorithm)
- Cache stampede protection built into modern Redis with SETNX + Lua scripts

sd-15 · Core Concepts

Explain database sharding — strategies, tradeoffs, and when to use it

SHARDING = horizontal partitioning
Split rows across multiple DB servers. Each shard holds a subset.
(Different from replication which COPIES all data)

SHARDING STRATEGIES
1. Hash-based: shard = hash(user_id) % N
✓ Even distribution
Adding shards = rehash everything (fix: consistent hashing)
2. Range-based: IDs 1–1M on shard 1, 1M–2M on shard 2
✓ Good for range queries
Hot spots: recent data gets all traffic
3. Directory-based: lookup table maps key → shard
✓ Flexible, easy to rebalance
Lookup service is a single point of failure

PROBLEMS SHARDING INTRODUCES
- Cross-shard joins: painful → denormalize or use application-side joins
- Cross-shard transactions: use Saga pattern, avoid if possible
- Hot shards: celebrity users get disproportionate load
- Rebalancing: consistent hashing minimises data movement

WHEN TO SHARD
- LAST RESORT — try these first:

1. Add read replicas
2. Add caching layer
3. Vertical scaling (bigger machine)
4. Then shard for write scaling only

WOW FACTOR
- Consistent hashing: add/remove shard = only K/n keys remap (not full reshuffle)
- Virtual nodes: each shard gets 150 ring positions = even load despite different hardware

sd-16 · Core Concepts

When do you use a message queue? How does Kafka work?

USE A MESSAGE QUEUE WHEN
- Decoupling: producer shouldn't wait for consumer to finish
- Traffic spikes: queue absorbs bursts, consumer processes at own pace
- Fan-out: one event → multiple consumers (email + SMS + push)
- Async: user doesn't need immediate result (report generation, emails)

KAFKA ARCHITECTURE
- Topics: named streams (like a table in a DB)
- Partitions: topic split into N partitions for parallelism
- Producers: write to topic, choose partition by key hash
- Consumers: read from offset (position), track progress themselves
- Consumer Groups: each partition assigned to ONE consumer → parallel processing

DELIVERY GUARANTEES
- At-most-once: fire and forget — possible data loss
- At-least-once: retry on failure → possible duplicates (make consumers idempotent)
- Exactly-once: Kafka transactions + idempotent consumers (harder, use sparingly)

KAFKA vs SQS
- Kafka: replay messages (seek to any offset), persistent, high throughput, complex
- SQS: managed, simpler, messages deleted after consume, no replay
- Use Kafka for event sourcing, audit logs, stream processing
- Use SQS for simple background job queues

WOW FACTOR
- Kafka as source of truth: events ARE the database, rebuild state by replaying
- Consumer lag: if consumers fall behind, queue grows → alert on lag metric
- Compacted topics: keep only latest value per key (acts like a KV store)

sd-17 · Core Concepts

Explain Consistent Hashing — why it matters for distributed systems

THE PROBLEM WITH NAIVE HASHING
- shard = hash(key) % N
- Add one server (N → N+1) → almost ALL keys remap to different shards
- Massive data migration every time you scale

CONSISTENT HASHING SOLUTION
- Place both servers AND keys on a circular ring (0 to 2^32)
- Key is served by the first server clockwise from its hash position
- Add a server → only the keys between it and its predecessor remaps
- Remove a server → only that server's keys move to the next one
- Adding/removing a node moves only K/N keys on average

VIRTUAL NODES (the practical improvement)
- Each physical server = 150+ positions on the ring
- Why: even small server clusters get uneven key distribution without this
- Result: much better load balancing, even if servers have different capacities
- Weighted assignment: powerful machines get more virtual nodes

REAL-WORLD USAGE
- Cassandra: consistent hashing for partition placement
- DynamoDB: consistent hashing with preference lists
- Redis Cluster: hash slots (16,384 slots) assigned to nodes
- CDN: route requests to nearest edge node

WOW FACTOR
- Hot spot problem: if many keys hash near one point → that server overloaded
- Fix: virtual nodes spread the hot zone across multiple physical servers
- Consistent hashing is also used in load balancers for session stickiness

sd-18 · Core Concepts

SQL vs NoSQL — how do you choose the right database?

SQL (Relational)
- ACID transactions, complex joins, strict schema, foreign keys
- Great for: financial data, user accounts, anything needing JOIN queries
- Examples: PostgreSQL, MySQL
- Scales vertically first, horizontal with sharding (complex)

NOSQL TYPES & USE CASES
Document Store (MongoDB, Firestore)
- Flexible schema, JSON documents, no joins
- Use: user profiles, product catalogs, content management

Key-Value (Redis, DynamoDB)
- Blazing fast lookups by key, no queries
- Use: sessions, caching, leaderboards, rate limiting

Wide Column (Cassandra, HBase)
- Write-heavy, time-series, massive scale
- Use: IoT data, chat messages, activity feeds, logs

Graph DB (Neo4j)
- Relationships are first-class, complex traversals
- Use: social networks, recommendation engines, fraud detection

DECISION FRAMEWORK
- Need complex queries + joins + transactions → SQL
- Need massive writes + horizontal scale → Cassandra
- Need sub-millisecond reads + caching → Redis
- Need flexible schema + document model → MongoDB

WOW FACTOR
- Polyglot persistence: most large systems use 3–4 different DBs for different needs
- Don't default to one → explain WHY each component uses a specific DB
- PostgreSQL can handle NoSQL too (JSONB columns) — often underestimated

sd-19 · Core Concepts

Explain the 8-Step System Design Interview Process

THE FRAMEWORK (use this structure every time)
① CLARIFY USE CASES
- List major features. Pick 2–3 to focus on.
- Ask: read-heavy or write-heavy? DAU? QPS? Global?

② ESTIMATE SCALE
- Back-of-envelope: storage/day, requests/sec, bandwidth
- 1M users × 10 req/day = ~116 req/sec

③ DEFINE DATA MODEL
- What entities exist? SQL or NoSQL? How does data flow?

④ ABSTRACT DESIGN
- Sketch: Client → LB → App Servers → Cache → DB
- Show the basic workflow first before going deep

⑤ DETAILED DESIGN
- Deep dive on 1–2 components. Explain trade-offs.
- Respond to interviewer probes — they're guiding you

⑥ IDENTIFY BOTTLENECKS
- Single points of failure? Hot partitions? Cache eviction?
- Ask: what breaks at 10x current scale?

⑦ SCALE THE DESIGN
- Add: read replicas, sharding, CDN, message queue, async workers

⑧ SHOW EXPERIENCE
- Mention industry best practices, real trade-offs you'd consider
- Reference real systems: "Twitter uses a hybrid fan-out approach because..."

WOW FACTOR
- Drive the conversation — interviewer should follow you, not lead
- Capacity estimation: order of magnitude is enough (TB vs GB, millions vs thousands)
- Senior+ tip: casually calculate exact numbers with cost analysis

sd-31 · Core Concepts

Explain DB Indexing and DB Replication

DB INDEXING
What it is: data structure (B+ tree or hash) that speeds up reads on a column
- B+ tree index: sorted, supports range queries (>, <, BETWEEN, ORDER BY)
- Hash index: exact match only, faster for equality (=)

When to index:
- Columns in WHERE, JOIN, ORDER BY, GROUP BY clauses
- Foreign key columns (avoid full table scans on joins)
- Avoid over-indexing: every index = slower writes + more storage

Cost of indexes:
- Reads: dramatically faster (O(log N) vs O(N) full scan)
- Writes: every INSERT/UPDATE/DELETE must also update all indexes
- Rule: index columns you filter/sort on, not columns you only select

DB REPLICATION
Master-Slave (Primary-Replica):
- All writes → master (primary)
- All reads → slaves (replicas) — scale read throughput horizontally
- Replication lag = slaves are slightly behind master
- Lag = eventual consistency for reads (may see stale data)

Synchronous replication:
- Write confirmed only after replica also writes → no data loss, slower writes

Asynchronous replication:
- Write confirmed at master, replica catches up → faster writes, potential data loss

WOW FACTOR
- Read-write splitting: route writes to master, reads to nearest replica
- Replication lag monitoring: alert if lag > 1 second (stale reads become a problem)
- Multi-master: both masters accept writes → conflict resolution needed (avoid if possible)
- Covering index: index includes all columns needed by query → zero table access needed

sd-32 · Core Concepts

Explain Bloom Filters — what they are and where they're used

WHAT IS A BLOOM FILTER?
A space-efficient probabilistic data structure that answers:
"Is this element definitely NOT in the set?" with 100% accuracy
"Is this element possibly in the set?" with configurable accuracy

HOW IT WORKS
- Bit array of M bits, initialized to 0
- K different hash functions
- ADD element: hash with all K functions → set those K bits to 1
- CHECK element: hash with all K functions → if ANY bit is 0 → definitely not in set
→ if ALL bits are 1 → probably in set

PROPERTIES
- False positives: possible (says "maybe yes" when actually "no")
- False negatives: IMPOSSIBLE (if it says "no", it's definitely no)
- Space: 1B elements, 1% false positive rate ≈ 1.2GB (vs 20GB for hash set)
- Cannot delete elements (use Counting Bloom Filter for deletion)

REAL-WORLD USES
- Cassandra/HBase: check if key exists in SSTable before expensive disk read
- Chrome Safe Browsing: check if URL is in malicious list (local bloom filter)
- Web Crawler: check if URL already crawled — avoid re-crawling
- CDN: check if content is cached at edge before going to origin
- Databases: skip disk lookup for keys that definitely don't exist

WOW FACTOR

- Tunable false positive rate: more bits = fewer false positives = more memory
- Formula: M = −N×ln(p) / (ln2)² where N=items, p=false positive rate
- Counting Bloom Filter: use counters instead of bits → supports deletion

sd-33 · Core Concepts

Explain CDN (Content Delivery Network) — how it works and when to use it

WHAT IS A CDN?

A globally distributed network of edge servers that cache content close to users.
Goal: reduce latency, reduce load on origin server, improve availability.

HOW IT WORKS

1. User requests resource (image, video, JS file)
2. DNS routes request to nearest CDN edge server (based on GeoDNS)
3. Edge server checks cache:
   - HIT → return cached content immediately (low latency)
   - MISS → fetch from origin → cache it → return to user
4. Next request for same content → served from edge (no origin hit)

CDN USE CASES

Static assets: JS, CSS, images, fonts → long TTL (1 week+)
Video streaming: HLS/DASH segments cached at edge → adaptive bitrate
API responses: GET endpoints with cache-control headers (short TTL)
Dynamic content: Edge computing (Cloudflare Workers) for personalisation at edge

CACHE INVALIDATION

- TTL-based: content expires after fixed time → simple but may serve stale content
- Purge API: immediately invalidate specific URLs (use after deploy)
- Versioned URLs: /static/app.v1.2.3.js → never expires (immutable)

WOW FACTOR

- Origin shield: one CDN PoP designated as origin shield → prevents thundering herd on origin
- Push vs Pull CDN:
- Pull: CDN fetches from origin on first miss (most common — lazy)
- Push: you proactively upload content to CDN (for large files, predictable access)
- Geographic compliance: CDN can route EU users to EU servers (GDPR)
- Cost: ~90% of bandwidth served from CDN at fraction of origin cost

sd-34 · Core Concepts

Pessimistic vs Optimistic Locking — when do you use each?

PESSIMISTIC LOCKING

"Assume conflict WILL happen — lock first, then read/write"

How it works:

- SELECT ... FOR UPDATE → locks the row immediately
- No other transaction can read or write that row until you commit/rollback
- Database-level lock

When to use:

- High contention: multiple users likely to update same record simultaneously
- Financial transactions: bank account balance, inventory count
- Short operations where you can't afford any conflict

Downsides:

- Deadlock risk: two transactions waiting for each other's locks
- Low throughput: locks block other readers
- Doesn't scale well for high-concurrency systems

OPTIMISTIC LOCKING

"Assume conflict is RARE — read freely, check at write time"

How it works:

- Add version column to table (version INT)
- Read record, note version (e.g. version=5)
- On update: WHERE id=? AND version=5 → if 0 rows updated → conflict → retry
- No database locks held during business logic

When to use:

- Low contention: users rarely update the same record simultaneously
- Long-running operations: can't hold lock for seconds
- Read-heavy workloads: many readers, few writers

WOW FACTOR

- Optimistic is default in most ORMs (Hibernate, Django ORM)
- For distributed systems: use version vectors or CRDTs instead of DB locks
- Saga pattern: avoids distributed locking entirely with compensating transactions

sd-35 · Core Concepts

Pull vs Push Model — SSE, WebSockets, Long-Polling explained

PULL MODEL (Client Polls)

Client periodically asks server: "Any updates?"

Short Polling:

- Client sends HTTP request every N seconds
- Simple to implement
- Wasteful: most responses are empty
- Latency: up to N seconds to see new data
- Use: low-frequency updates, simple dashboards

Long Polling:

- Client sends request → server holds connection open until update or timeout
- Better latency, still HTTP-based (works everywhere)
- Server must handle many hanging connections
- Use: chat apps (before WebSockets), notifications

PUSH MODEL (Server Notifies)

Server sends data to client when it's available

Server-Sent Events (SSE):

- One-way: server → client only
- Persistent HTTP connection, text-based
- Auto-reconnect built in
- Use: live feeds, stock prices, notifications

WebSocket:

- Bi-directional: both sides can send/receive
- Full-duplex over single TCP connection
- Lower overhead than HTTP for frequent messages
- Use: chat, collaborative editing, multiplayer games, live tracking

WOW FACTOR

- WebSocket connection per user → connection service maps user_id → server in Redis
- Horizontal scaling: sticky sessions (IP hash LB) OR move connection state to Redis pub/sub
- SSE vs WebSocket: SSE simpler for one-way push, WebSocket when client also sends frequently
- Fallback strategy: WebSocket → SSE → Long Poll (decreasing capability, increasing compatibility)

sd-36 · Core Concepts

Availability Patterns — Active-Passive, Active-Active, and Chaos Engineering

ACTIVE-PASSIVE (Fail-Over)

- One active server handles all traffic
- Passive standby is ready but idle
- On failure: DNS switches or VIP floats to passive → becomes new active
- Cold standby: passive not running (cost-efficient, slower failover ~minutes)
- Warm standby: passive running but not serving (faster failover ~seconds)
- Hot standby: passive also processing (near-instant failover)
- Use: databases, stateful services where split-brain is dangerous

ACTIVE-ACTIVE

- Multiple servers ALL handle traffic simultaneously
- Load balanced across all nodes
- On failure: remaining nodes absorb traffic (capacity pre-provisioned for this)
- Requires stateless design (or shared state in Redis/DB)
- Use: web app servers, API gateways, caches

AVAILABILITY MATH

- 99% = 3.65 days downtime/year
- 99.9% (3 nines) = 8.77 hours/year
- 99.99% (4 nines) = 52.6 minutes/year
- 99.999% (5 nines) = 5.26 minutes/year → very expensive to achieve

REPLICATION FOR DURABILITY

- N=3 replicas: can lose 2 and still serve data
- Geographic distribution: survive entire data center failure

WOW FACTOR

- Chaos Engineering (Netflix Chaos Monkey): deliberately kill random servers in production
- Forces team to build true resilience — not just paper-resilience
- Circuit breakers + bulkheads: limit blast radius when failure does occur
- SLA vs SLO vs SLI: define what availability means before designing for it

sd-37 · Core Concepts

HyperLogLog, Merkle Tree, and LSM Tree explained

HYPERLOGLOG

Problem: Count distinct items in a stream with billions of elements
Naive: store every element → too much memory

How it works:

- Probabilistic algorithm, ~0.81% error rate
- Uses ~1.5KB memory regardless of cardinality
- Hashes elements, observes patterns in leading zeros

Use cases:

- Redis PFCOUNT: count unique visitors per page
- Unique search queries per day
- Distinct IPs hitting an endpoint

MERKLE TREE

Problem: Efficiently compare two large datasets to find differences

How it works:

- Each leaf = hash of a data block
- Each parent = hash of its children
- Root hash = fingerprint of entire dataset
- Two datasets differ → compare root hashes → traverse to find exactly which blocks differ

Use cases:

- Git: detects which files changed between commits
- Cassandra: anti-entropy repair (sync diverged replicas efficiently)
- Blockchain: verify transaction integrity in a block

LSM TREE (Log-Structured Merge Tree)

Problem: High write throughput — random disk writes are slow

How it works:

- Writes go to in-memory MemTable (sorted) → fast
- When full: flush MemTable to disk as SSTable (immutable, sorted)
- Background: merge/compact SSTables to reduce read amplification

Use cases: Cassandra, RocksDB, LevelDB, HBase (all write-heavy systems)

WOW FACTOR

- LSM vs B-tree: LSM = fast writes, slower reads. B-tree = fast reads, slower writes.
- Bloom filter + LSM: filter eliminates most unnecessary SSTable reads

sd-55 · Core Concepts

Explain Networking Fundamentals — OSI model, DNS, TCP vs UDP, Proxy vs Reverse Proxy

CLARIFY FIRST

These are the foundation interviewers expect you to know cold.

OSI MODEL (7 layers — remember: "Please Do Not Throw Sausage Pizza Away")

7. Application — HTTP, FTP, SMTP, DNS
6. Presentation — encryption, compression, encoding (TLS)
5. Session — session management, auth tokens
4. Transport — TCP (reliable) / UDP (fast) — ports live here
3. Network — IP addresses, routing between networks
2. Data Link — MAC addresses, LAN communication
1. Physical — cables, wifi, fiber

DNS (Domain Name System)

Browser → Recursive Resolver → Root NS → TLD NS → Authoritative NS → IP

- A record: domain → IPv4
- CNAME: alias to another domain (www → example.com)
- TTL: how long to cache before re-querying
- DNS load balancing: return different IPs per region (GeoDNS)

TCP vs UDP

TCP: connection-oriented, 3-way handshake, guaranteed delivery, ordered
→ Use for: HTTP, file transfers, anything where accuracy > speed
UDP: connectionless, fire-and-forget, no retransmit
→ Use for: video streaming, gaming, DNS, live audio (latency > reliability)

PROXY vs REVERSE PROXY

Forward Proxy: sits in front of CLIENTS — hides client identity, VPNs
Reverse Proxy: sits in front of SERVERS — load balancing, SSL termination, caching, hides server topology

- Nginx / HAProxy / Cloudflare are reverse proxies
- "Client sees one IP; server could be 1000 machines behind it"

WOW FACTOR

- HTTP/3 runs over QUIC (UDP-based) — Google's fix for TCP head-of-line blocking
- DNS over HTTPS (DoH): encrypts DNS queries — prevents ISP snooping
- CDNs are essentially a globally-distributed reverse proxy fleet

sd-56 · Core Concepts

Explain ACID Transactions and when to use them

CLARIFY FIRST

ACID = guarantees for database transactions. Critical for financial and inventory systems.

ACID BREAKDOWN

A — Atomicity
All operations in a transaction succeed or all fail. No partial writes.
→ "Transfer $100: debit Alice AND credit Bob — if credit fails, debit is rolled back"

C — Consistency
Every transaction takes the DB from one valid state to another. Constraints are never violated.
→ Account balance can never go negative (if constraint defined)

I — Isolation
Concurrent transactions don't see each other's intermediate state.
Isolation levels (weakest to strongest):
- Read Uncommitted: can read dirty (uncommitted) data — fastest, dangerous
- Read Committed: only read committed data — default in most DBs
- Repeatable Read: same row read twice returns same value in a transaction
- Serializable: transactions behave as if sequential — safest, slowest

D — Durability
Committed data survives crashes. Achieved via Write-Ahead Log (WAL).

WHEN TO USE

Use ACID: payments, banking, inventory, bookings, order management
Relax to BASE (Eventually Consistent): social feeds, analytics, likes/views, search indexes

DISTRIBUTED TRANSACTIONS

Single DB: ACID is "free" (DB handles it)
Multi-service: use Saga Pattern (chain of local transactions + compensations)
or Two-Phase Commit (coordinator + participants — blocking, avoid if possible)

SCALE TRADEOFFS

ACID with Serializable isolation → throughput drops 10-100x
Most systems use Read Committed + application-level locking for hot rows

WOW FACTOR

- PostgreSQL WAL: every write goes to WAL first → crash recovery replays log
- "Phantom reads" — Repeatable Read doesn't prevent new rows matching a query from appearing
- CockroachDB and Spanner offer distributed ACID with global consistency via atomic clocks

sd-57 · Core Concepts

Explain Event-Driven Architecture and Microservices

CLARIFY FIRST

When to choose event-driven vs request/response, and how microservices fit in.

EVENT-DRIVEN ARCHITECTURE

Components:
- Producer: publishes events (e.g., "OrderPlaced")
- Event Bus: Kafka / RabbitMQ / SNS+SQS
- Consumer: subscribes to events, processes async

Benefits:

Loose coupling — producer doesn't know consumers

Scalability — consumers scale independently

Resilience — consumer failure doesn't break producer

Audit trail — event log is a history of everything

MICROSERVICES vs MONOLITH

Monolith pros: simple to deploy, no network latency, easy transactions
Monolith cons: one failure can cascade, harder to scale individual parts
Microservices pros: independent deployment, technology diversity, team ownership
Microservices cons: distributed systems complexity, network failures, data consistency hard

WHEN TO USE EACH

Event-driven (async): notifications, sending emails, updating search indexes, analytics
Request/response (sync): user-facing APIs where you need the result immediately
Common patterns:
- Choreography: services react to events independently (no coordinator)
- Orchestration: central saga orchestrator tells each service what to do

OPERATIONAL CONCERNS

Service discovery: how do services find each other? (Consul, Kubernetes DNS)
Distributed tracing: track a request across services (Jaeger, Zipkin, Datadog)
Circuit breaker: stop calling failing services (Hystrix, Resilience4J)

WOW FACTOR

- Netflix: 1000+ microservices, event-driven via Kafka, coordinated by Spring Cloud
- "Outbox pattern": write event to DB table in same transaction, then publish → ensures exactly-once event delivery
- Event sourcing: store events as the source of truth, derive current state by replaying

sd-58 · Core Concepts

Explain Change Data Capture (CDC), Pub/Sub, and Message Queues

CLARIFY FIRST

These are the plumbing of async distributed systems.

MESSAGE QUEUES

Point-to-point: one producer, one consumer, message deleted after consumption
Examples: SQS, RabbitMQ
Use for: task queues, job distribution, rate limiting work

PUB/SUB

One producer, many consumers (fan-out), each consumer gets a copy
Examples: Kafka (persistent log), Google Pub/Sub, SNS+SQS
Use for: event broadcasting, activity feeds, notifications to many services

Key difference: Queue = each message processed once. Pub/Sub = each subscriber gets every message.

KAFKA DEEP DIVE

- Topics split into Partitions — parallelism unit
- Consumer Groups: each group gets all messages; within group, each partition → one consumer
- Offset: position in partition, consumers commit after processing
- Retention: messages kept for configurable period (default 7 days) — replay possible
- At-least-once delivery: may get duplicates → consumers must be idempotent

CHANGE DATA CAPTURE (CDC)

What: capture every insert/update/delete from DB and stream as events
How: Debezium reads PostgreSQL WAL (write-ahead log) and publishes to Kafka
Why: sync DB changes to search indexes, caches, analytics without polling

CDC flow:
DB write → WAL entry → Debezium reads WAL → Kafka topic → Elasticsearch / Redis / DW

PATTERNS

Exactly-once: Kafka transactions + idempotent consumers
Dead Letter Queue (DLQ): failed messages go to separate queue for inspection
Backpressure: consumer signals producer to slow down when overwhelmed

WOW FACTOR

- LinkedIn built Kafka — original use case was tracking user activity events at scale
- CDC is how you keep Elasticsearch in sync with PostgreSQL without dual-writes
- "Log is the database": Kafka's persistent log is itself a source of truth — replay to rebuild any derived store

sd-59 · Core Concepts

Explain Idempotency, API Design best practices, and WebSockets vs REST

CLARIFY FIRST

API design decisions affect reliability and client experience at scale.

IDEMPOTENCY

Definition: calling the same operation multiple times has the same effect as calling it once.
Critical for: payments, order creation, anything that can be retried

Implementation:
- Client sends Idempotency-Key header (UUID per request)
- Server stores key → response in Redis/DB with TTL
- On retry with same key: return cached response, don't re-execute
- GET, HEAD, PUT, DELETE are naturally idempotent. POST is NOT — must add idempotency key.

REST vs GraphQL vs gRPC

REST:
- Simple, cacheable, stateless
- Over-fetching: client gets fields it doesn't need
- Under-fetching: multiple calls to get related data
- Use for: public APIs, mobile clients, standard CRUD

GraphQL:
- Client specifies exactly what fields it needs
- Single endpoint, flexible queries
- Harder to cache, schema overhead
- Use for: complex nested data, multiple client types (mobile/web) with different needs

gRPC:
- Binary protocol (Protocol Buffers), low latency
- Bi-directional streaming
- Use for: internal microservice communication, real-time data, IoT

WEBSOCKETS vs REST

REST: request/response, stateless, client initiates
WebSocket: persistent TCP connection, bidirectional, server can push
Use WebSocket for: live chat, real-time collaboration, live sports scores, stock tickers

Alternatives:
- Long Polling: client polls repeatedly, server holds until data ready — simple but inefficient
- Server-Sent Events (SSE): one-way push from server, simpler than WebSocket

RATE LIMITING

Token bucket: X tokens replenish/sec, each request costs 1 token — handles bursts
Sliding window: count requests in rolling window — more accurate
Fixed window: simple counter per time window — susceptible to boundary bursts

WOW FACTOR

- Stripe uses idempotency keys on every payment API — prevents double-charge on retry
- HTTP/2 multiplexing: multiple requests over one connection — makes REST competitive with gRPC for web
- Redis SETNX (set-if-not-exists) is the primitive for distributed idempotency key storage

sd-60 · Core Concepts

Explain Fault Tolerance, Failover, and Disaster Recovery

CLARIFY FIRST

How to design systems that stay up when things go wrong — and things always go wrong.

KEY CONCEPTS

Single Point of Failure (SPOF): any component whose failure takes down the whole system
→ Solution: eliminate SPOFs by adding redundancy at every layer

Fault Tolerance: system continues operating (possibly degraded) despite failures
Fault Isolation: failures don't cascade — one failing service doesn't kill others

High Availability (HA): expressed as uptime percentage:
- 99% = 87.6 hrs downtime/year
- 99.9% = 8.76 hrs/year
- 99.99% = 52.6 min/year
- 99.999% = 5.26 min/year (five nines)

REDUNDANCY PATTERNS

Active-Passive: primary handles traffic, secondary on standby — fast failover, wastes resources
Active-Active: both handle traffic, load balanced — better utilization, more complex
Multi-region: replicas across geographic regions — survives data center outage
Multi-AZ: replicas across availability zones — survives rack/power failure

FAILOVER STRATEGIES

Health checks: load balancer polls /health endpoint every 5s
Circuit breaker: after N failures, stop calling failing service for X seconds
Retry with exponential backoff: retry(1s) → retry(2s) → retry(4s) → DLQ
Graceful degradation: return cached/stale data instead of error
Bulkhead: isolate failures — thread pool per dependency, one slow DB can't block all requests

DISASTER RECOVERY

RPO (Recovery Point Objective): max data loss acceptable ("we can lose up to 1 hour of data")
RTO (Recovery Time Objective): max downtime acceptable ("we must be back in 15 minutes")
Backup strategies: full backup daily, incremental hourly, WAL streaming for near-zero RPO

WOW FACTOR

- Netflix Chaos Engineering: intentionally kill random servers in production — if you don't test failure, it will surprise you
- "Design for failure" principle: assume every network call fails, every disk dies
- AWS S3: 11 nines (99.999999999%) durability — achieved by storing objects across 3+ AZs with erasure coding

Cloud Patterns · 6 cards

sd-20 · Cloud Patterns

Explain Circuit Breaker, Bulkhead, and Retry patterns

CIRCUIT BREAKER

Problem: one slow service causes cascading failures across the whole system
- CLOSED: requests flow normally, count failures
- OPEN: after N failures, stop all requests to that service immediately
- HALF-OPEN: after timeout, let one request through to test recovery
- Real example: Netflix Hystrix, Resilience4j

BULKHEAD

Problem: one slow service exhausts all threads → crashes everything else
- Isolate services into separate thread pools (like ship bulkhead compartments)
- Service A gets 20 threads, Service B gets 20 threads — B failure can't starve A
- Apply to: DB connection pools, HTTP client pools

RETRY WITH EXPONENTIAL BACKOFF

Problem: transient failures — don't retry immediately (thundering herd)
- Retry after: 1s → 2s → 4s → 8s → give up
- Add jitter (random delay): prevents all clients retrying in sync
- Only retry idempotent operations (GET, PUT with idempotency key)

TIMEOUT (the often-forgotten one)
- Always set timeouts — never wait forever for a dependency
- Fail fast: better to return error quickly than hang all resources

WOW FACTOR
- Use ALL THREE together: timeout → retry → circuit breaker → bulkhead
- Without them: 1 bad microservice takes down your entire platform
- Chaos engineering (Netflix's approach): deliberately break things to find gaps

sd-21 · Cloud Patterns

Explain the Saga Pattern for distributed transactions

THE PROBLEM

Distributed transactions across multiple services — you can't use traditional ACID.
A two-phase commit (2PC) is blocking and fragile in microservices.

SAGA PATTERN

Break a distributed transaction into a sequence of local transactions.
Each step publishes an event that triggers the next.
If any step fails → run compensating transactions in reverse.

EXAMPLE: E-commerce Order
✓ Place Order → Charge Payment → Reserve Inventory → Ship → Notify
If inventory fails:
Cancel notification ← Release inventory ← Refund payment ← Cancel order

TWO SAGA STYLES
Choreography (event-driven, no central coordinator):
- Each service listens for events and reacts
- Pro: loosely coupled, simple for small flows
- Con: hard to track overall state, debugging complex

Orchestration (central saga coordinator):
- Saga Orchestrator tells each service what to do next
- Pro: clear flow, easy to monitor, centralized error handling
- Con: orchestrator can become a bottleneck

WHEN TO USE
- Multi-service transactions: order + payment + inventory + notification
- Long-running business processes where eventual consistency is acceptable

WOW FACTOR
- Combine with idempotency keys: safe to retry any step without side effects
- Event sourcing: Saga events become your audit log automatically
- Outbox pattern: write event to DB and queue atomically → no lost messages

sd-22 · Cloud Patterns

What is the difference between horizontal and vertical scaling? When do you use each?

VERTICAL SCALING (Scale Up)
- Give one machine more CPU, RAM, faster disk
- Pro: simple — no code changes, no distributed system complexity
- Con: hard limit (biggest machine available), single point of failure
- Use: start here first. Always try vertical before horizontal.

HORIZONTAL SCALING (Scale Out)
- Add more machines, distribute load across them
- Pro: theoretically unlimited scale, no single point of failure
- Con: app must be stateless, need load balancer, distributed system complexity
- Use: when vertical hits its limit, or for geographic distribution

STATELESS DESIGN (requirement for horizontal scaling)
- No user session stored on server — store in Redis or JWT
- Any server can handle any request → LB can route freely

TOOLS FOR HORIZONTAL SCALING
- Load Balancer: Round-robin, least connections, IP hash for stickiness
- Auto-scaling groups: spin up/down instances based on CPU/request metrics
- Read replicas: horizontally scale reads from DB
- Sharding: horizontally scale DB writes (last resort)

REAL DECISION FRAMEWORK
1. Is the app stateless? (If no, fix this first)
2. Can we cache more aggressively? (Often solves 80% of load)
3. Can we add a read replica? (Solves read-heavy workloads)
4. Vertical scale the DB? (Usually cheaper than sharding)
5. Then horizontal scale app servers + shard DB

WOW FACTOR
- Cattle vs Pets: treat servers as replaceable cattle, not named pets
- Immutable infrastructure: replace servers on deploy, never patch in-place
- Auto-healing: health checks + auto-replace unhealthy instances

sd-43 · Cloud Patterns

Common System Design Mistakes to Avoid

MISTAKE 1: Jumping into solution before clarifying
"I'll design this as a microservices system with Kafka..."

Fix: Always ask 2-3 clarifying questions first. DAU? QPS? Read vs write heavy? Global or regional?
MISTAKE 2: Not driving the conversation
Interviewer is asking all the questions, you're just answering.

Fix: You lead. Say "Let me start with requirements, then data model, then architecture."
MISTAKE 3: Generic answers with no concrete trade-offs
"We'll use a database and a cache."

Fix: "We'll use Cassandra because it's write-heavy time-series data, partitioned by (user_id, month). Redis for hot read cache."
MISTAKE 4: Stubbornly defending your design
Interviewer suggests an alternative → you dismiss it.

Fix: "That's a great point — if we used X instead, it would give us Y but we'd lose Z. For this use case, I'd still prefer A because..."
MISTAKE 5: Skipping capacity estimation
Jumping to architecture without knowing if you need 100 QPS or 1M QPS.

Fix: 2 minutes of back-of-envelope math. TB vs GB? Millions vs thousands of QPS?
MISTAKE 6: Not mentioning failure scenarios
Only designing for the happy path.

Fix: "What happens if this service goes down? We'd add a circuit breaker + retry with backoff."
MISTAKE 7: Ignoring schema design
Handwaving "we'll store data in a database"

Fix: Explicitly state the key tables/documents with important columns. Interviewers judge architecture by data model quality.

WOW FACTOR
- Show adaptability: "I'd choose X, but Y is also valid if the requirements shift toward..."
- Reference real systems: "Twitter had this exact problem and solved it with hybrid fan-out"

sd-44 · Cloud Patterns

Cloud Design Principles — all 8 you must know

1. FAIL FAST
Return error immediately rather than hanging. Timeout aggressively.
"A request that fails in 100ms is better than one that hangs for 30 seconds"

2. DESIGN FOR FAILURE
Assume EVERY component will fail. Build with retries, circuit breakers, bulkheads.
No single points of failure. Multi-AZ deployments.

3. IMMUTABLE INFRASTRUCTURE
Never patch servers in-place. Replace them. Use containers/AMIs/images.
Eliminates configuration drift. "Snowflake servers" are a liability.

4. CATTLE vs PETS
Pets: named servers you care about (db01, web-prod) — you nurse them when sick
Cattle: numbered, replaceable, auto-replaced on failure (pod-7842 dies → new one spins up)
Design for cattle.

5. AUTO HEALING
Health checks on every service. Auto-restart/replace on failure.
Kubernetes liveness probes. Load balancer health checks. Auto Scaling Groups.

6. ASYNC PROGRAMMING
Non-blocking I/O. Message queues for decoupling services.
Never block on I/O. Event-driven > synchronous request chains.

7. GITOPS
Infrastructure as code (Terraform, CloudFormation). Version controlled. Reviewed. Audited.
No manual changes. Every change = a pull request. Full audit trail.

8. EVENT-DRIVEN ARCHITECTURE
Services communicate via events, not direct calls.
Producer doesn't know about consumers. Loose coupling.
Example: OrderPlaced event → InventoryService, EmailService, AnalyticsService all react independently.

WOW FACTOR
- Chaos engineering: deliberately break things (Netflix Chaos Monkey) to find real weaknesses
- Observability: logs + metrics + traces (the three pillars). If you can't observe it, you can't fix it.

sd-45 · Cloud Patterns

Cloud Design Patterns — Ambassador, Cache-Aside, Gateway Aggregation, Priority Queue, Strangler Fig

AMBASSADOR
Helper sidecar service handles cross-cutting concerns beside the main service.
- Main service focuses on business logic
- Ambassador handles: retries, circuit breaking, monitoring, TLS, service discovery
- Example: Envoy proxy as sidecar in Kubernetes / Istio service mesh
- Use: when you want to add network capabilities without changing app code

CACHE-ASIDE (Lazy Loading)
App manages cache explicitly:
1. Check cache → HIT → return
2. MISS → fetch from DB → write to cache → return
- App controls what gets cached and when
- Use: general caching, when you need fine-grained control over cache population

GATEWAY AGGREGATION
Client needs data from 5 microservices → Gateway makes 5 calls → returns 1 combined response
- Reduces chattiness between client and backend
- Client makes 1 request instead of 5 → less latency on mobile
- Gateway can cache, transform, and merge responses
- Use: BFF (Backend For Frontend) pattern, mobile API layers

PRIORITY QUEUE
Different SLAs for different request types → separate queues per priority level
- Premium users: dedicated high-priority queue, processed first
- Free users: standard queue
- Critical alerts: separate queue never gets starved by bulk operations
- Implementation: multiple Kafka topics or SQS queues with different consumer ratios

STRANGLER FIG PATTERN
Incrementally replace a legacy monolith with microservices:
1. Put a facade/proxy in front of the monolith
2. Route new features to new microservices
3. Gradually migrate existing features one at a time
4. Old monolith "strangles" and eventually disappears
- Use: migrating legacy systems without big-bang rewrites

WOW FACTOR
- Ambassador in practice: Linkerd/Envoy are Ambassador pattern at scale
- Strangler Fig: how Netflix migrated from monolith to microservices over years

Advanced Data Structures · 5 cards

sd-38 · Advanced Data Structures

Gossip Protocol and Vector Clocks in distributed systems

GOSSIP PROTOCOL
Problem: How do N nodes share state without a central coordinator?
How it works:
- Each node periodically picks K random neighbors and shares its known state
- Recipients merge info and forward to their random neighbors
- Information propagates exponentially: O(log N) rounds to reach all nodes

Properties:
- Decentralized: no single point of failure
- Eventually consistent: all nodes converge on same view
- Resilient: nodes can fail, info still propagates
- Used in: Cassandra (cluster membership), DynamoDB, Redis Cluster

What gets gossiped:
- Which nodes are alive/dead (failure detection)
- Token assignments (which node owns which data range)
- Schema changes, configuration

VECTOR CLOCKS
Problem: In distributed systems, wall clock time can't determine causation
How it works:
- Each node maintains a vector: [node_A_count, node_B_count, node_C_count]
- On local event: increment own counter
- On message send: attach current vector
- On message receive: merge (take max of each position) + increment own

Example: A=[1,0,0], B=[0,1,0] → concurrent (neither happened before the other)
A=[1,0,0], B=[2,1,0] → B happened after A (B's vector dominates)
Use cases: DynamoDB conflict detection, CRDTs, distributed version control

WOW FACTOR
- Gossip for failure detection: if node not heard from in 3 gossip rounds → suspected dead
- Vector clocks grow with number of nodes → Dynamo uses them selectively
- Physical clocks: Google Spanner uses atomic clocks + GPS for true global ordering

sd-39 · Advanced Data Structures

CRDTs, Two-Phase Commit (2PC), and Paxos/Raft

CRDTs (Conflict-Free Replicated Data Types)

Problem: How to merge concurrent edits from multiple nodes without coordination?

How it works:

- Special data types designed to always merge deterministically
- G-Counter: each node has its own counter, value = sum of all. Only increments.
- PN-Counter: positive + negative counters → supports increment/decrement
- OR-Set: add-wins set. Each element has unique tag. Remove only removes tagged version.
- Text (RGA): sequence of characters with position identifiers → Google Docs style

Use cases: Figma (collaborative design), Notion, Redis CRDT, shopping carts

TWO-PHASE COMMIT (2PC)

Problem: How to ensure all nodes in a distributed transaction commit or all rollback?

Phase 1 (Prepare):

- Coordinator asks all participants: "Can you commit?"
- Each participant locks resources, writes to WAL, votes Yes or No

Phase 2 (Commit/Abort):

- If ALL voted Yes → Coordinator sends Commit to all
- If ANY voted No → Coordinator sends Abort to all

Problems:

- Blocking: if coordinator crashes after Phase 1 → participants stuck holding locks
- Use Paxos/Raft for coordinator election + recovery

PAXOS / RAFT (Consensus Protocols)

Problem: How do distributed nodes agree on a single value despite failures?

Raft (simpler to understand):

- Elect one leader → leader handles all writes
- Leader sends log entries to followers → majority must ack before commit
- On leader failure → election: node with most up-to-date log wins
- Used in: etcd (Kubernetes), CockroachDB, TiKV

WOW FACTOR

- Raft vs Paxos: same guarantees, Raft designed to be understandable (Diego Ongaro's thesis)
- 2PC vs Saga: 2PC = synchronous, locks. Saga = async, compensating. Saga preferred in microservices

sd-40 · Advanced Data Structures

Geohash and Quadtree for spatial/location-based systems

GEOHASH

Problem: How do you index and query by geographic location efficiently?

How it works:

- Encode latitude/longitude into a base32 string
- Longer string = smaller area (higher precision)
- Nearby locations share a common prefix

Examples:

- "dp3" = large area around Washington DC area
- "dp3wjz" = specific city block in DC
- Precision 5 chars = ~4.9km × 4.9km area

Use cases:

- Find nearby drivers: GEORADIUS in Redis (uses geohash internally)
- Yelp: find restaurants near user's location
- Partitioning: assign cells to servers based on geohash prefix

Limitation:

- Edge case: two points very close but different geohash prefixes (cross boundary)
- Fix: check 8 surrounding cells as well as target cell

QUADTREE

Problem: Efficiently index 2D space for arbitrary density distributions

How it works:

- Start with bounding rectangle covering entire area
- If a region has more than N points → subdivide into 4 quadrants
- Repeat until each leaf has ≤ N points (e.g. N=100 for Uber drivers)
- Variable cell sizes: dense cities = small cells, rural = large cells

Operations:

- Point query: O(log N) — traverse tree from root
- Range query: traverse all overlapping quadrants

Use cases: Uber driver matching, game collision detection, Google Maps, Yelp

WOW FACTOR

- Geohash vs Quadtree: geohash = fixed grid, quadtree = adaptive to density
- S2 Geometry (Google): spherical geometry library — handles Earth's curvature properly
- Uber uses S2: divides sphere into cells, handles poles and date line correctly

sd-41 · Advanced Data Structures

Skip List, Ring Buffer, Reservoir Sampling, and Segment Tree

SKIP LIST

- Sorted linked list with express lanes at multiple levels
- Search: start at highest level, drop down when overshoot → O(log N)
- Alternative to balanced BST — simpler to implement concurrently
- Redis ZADD (sorted sets) uses skip list internally
- Use: leaderboards, ordered indexes, priority queues

RING BUFFER (Circular Buffer)

- Fixed-size array with head and tail pointers that wrap around
- Producer writes at tail, consumer reads from head
- When full: overwrite oldest (or block, depending on use case)
- O(1) read and write, zero allocation after init
- Use: network I/O buffers, log ringbuffers (Disruptor), audio/video streaming

RESERVOIR SAMPLING

Problem: Sample K items from a stream of unknown size N, each with equal probability

- Initialize reservoir with first K items
- For item i (i > K): keep with probability K/i
- If kept: replace a random item in reservoir
- Result: each of N items has exactly K/N probability of being in final sample
- Use: sample from huge logs, A/B test random user selection, database TABLESAMPLE

SEGMENT TREE

- Binary tree where each node stores aggregate (sum, min, max) of a range
- Leaf nodes: individual elements
- Internal nodes: aggregate of children
- Range query (e.g. sum of indices 3–7): O(log N)
- Range update: O(log N) — lazy propagation
- Use: leaderboards (rank of score in range), scheduling (free time slots), analytics dashboards

WOW FACTOR

- Segment tree vs BIT (Binary Indexed Tree/Fenwick): BIT simpler, only prefix sums. Segment tree handles any range.
- Ring buffer in kernel: Linux kernel uses ring buffers for network packet queues

sd-42 · Advanced Data Structures

SSTable, Cuckoo Hashing, DHT, and Trie in system design context

SSTABLE (Sorted Strings Table)

- Immutable, sorted key-value file on disk (output of LSM Tree flush)
- Binary search to find a key: O(log N) — or O(1) with sparse index in memory
- Compaction: multiple SSTables merged into one (remove deleted/old values)
- Bloom filter per SSTable: skip reading file if key definitely not in it
- Use: Cassandra, LevelDB, RocksDB — all LSM-tree based systems

CUCKOO HASHING

- Two hash tables, two hash functions h1 and h2
- Lookup: always check exactly two positions → O(1) worst case (unlike chaining = O(N) worst)
- Insert: place at h1(key). If occupied, evict that element to its h2 position. Repeat.
- If cycle detected → rehash entire table
- Use: network hardware lookup tables, packet classification, routers (need guaranteed O(1))

DHT (Distributed Hash Table)

- Decentralized key lookup across N nodes — no central directory
- Kademlia protocol (BitTorrent, IPFS):
- Keys and nodes share the same 160-bit ID space
- Each node knows about nodes at exponentially increasing distances
- Lookup: O(log N) hops to find the node responsible for a key
- Use: BitTorrent (find peers with a file), IPFS (content addressing), P2P systems

TRIE (in system design)

- Prefix tree: each edge = one character, each node = prefix
- O(L) for insert/search where L = key length (not N = dataset size)
- Space: shared prefixes → efficient for large common-prefix datasets
- System design uses:
- Autocomplete: traverse to prefix node, return all children
- IP routing: longest prefix match for network routes
- DNS: hierarchical domain name lookup
- Spell check: find all words within edit distance 1

WOW FACTOR

- Trie in Redis: not native, but simulate with sorted sets or Redis Search module
- Cuckoo filter: like Bloom filter but supports deletion + slightly better space efficiency

Tradeoffs · 5 cards

sd-61 · Tradeoffs

Batch vs Stream Processing — when do you use each?

CLARIFY FIRST

One of the most common system design tradeoff questions — especially for analytics, ETL, and data pipelines.

BATCH PROCESSING

Process large volumes of data at scheduled intervals

Examples: Spark, Hadoop MapReduce, Airflow pipelines

Use when: results don't need to be real-time, data is bounded, cost efficiency matters

Pros: simple, high throughput, easy to re-run for corrections, cheap

Cons: latency (hours to days), stale results, large resource spikes at job time

Common uses: daily reports, ML model training, billing summaries, ETL to data warehouse, email digests

STREAM PROCESSING

Process data continuously as it arrives, record by record or in micro-batches

Examples: Apache Kafka Streams, Apache Flink, Spark Streaming, AWS Kinesis

Use when: near real-time results required (seconds not hours)

Pros: low latency, always fresh results, can detect patterns in real-time

Cons: complex (exactly-once semantics hard), state management, harder to debug

Common uses: fraud detection, real-time dashboards, live recommendations, IoT alerts, activity feeds

LAMBDA ARCHITECTURE

Combine both: batch layer for accuracy + speed layer for freshness

- Batch: recompute everything nightly (ground truth)
- Speed: stream process recent data for near-real-time views
- Query: merge both layers

Downside: maintain two systems — complex

Kappa Architecture: stream only, replay historical data through same pipeline for corrections — simpler

DECISION FRAMEWORK

Latency needed < 1 minute → Stream

Latency OK in hours → Batch

Need both accuracy and freshness → Lambda/Kappa

WOW FACTOR

- Flink's "watermarks" handle out-of-order events in streams — key insight for real-time aggregations
- Kafka's replayability means you can re-process all historical events through a new stream job — no separate batch system needed
- Uber processes 1T+ events/day with Flink for surge pricing and driver ETAs

sd-62 · Tradeoffs

Stateful vs Stateless Design — tradeoffs and when to use each

CLARIFY FIRST

Statefulness vs statelessness affects scalability, fault tolerance, and routing complexity.

STATELESS SERVICES

No per-user state stored in the server process itself.

Every request contains all information needed (or state is fetched from shared store).

Pros:

Any server can handle any request → easy horizontal scaling

Server failures don't lose user data (state is in DB/cache)

Simple load balancing (round-robin works)

Easy to deploy, roll back, restart

Cons: every request must fetch state from DB/cache → latency + load

Examples: REST APIs, static file servers, stateless microservices

STATEFUL SERVICES

Server holds per-user state in memory between requests.

Pros:

No DB roundtrip for state → lower latency

Rich session context (WebSocket connections, game state)

Cons:

"Sticky sessions" required — same user must hit same server

Server crash = lost state

Harder to scale — can't freely add servers

Rolling deploys break sessions

Examples: WebSocket servers, multiplayer game servers, live video encoding pipelines

HYBRID APPROACH (most real systems)

Stateless application layer + stateful data layer:

- App servers: stateless (scale freely)
- Redis: shared state (session data, counters, rate limits)
- PostgreSQL: durable state (orders, users)
- Kafka: event state (message log)

SESSION MANAGEMENT

JWT tokens: stateless auth — server validates signature, no DB lookup

Server-side sessions: stored in Redis, session ID in cookie — revocable

WOW FACTOR

- Kubernetes prefers stateless — StatefulSets are a workaround for databases
- "Share nothing" architecture: each server is fully independent — Pinterest's model for massive scale
- WebSocket gateway: one stateful layer at the edge, everything behind it stateless

sd-63 · Tradeoffs

Strong vs Eventual Consistency — CAP theorem in practice

CLARIFY FIRST

The most important tradeoff in distributed systems. Know when you need strong vs eventual.

STRONG CONSISTENCY

After a write completes, all reads return the new value immediately.

Every node sees the same data at the same time.

Cost: latency (must wait for all replicas to confirm) + reduced availability during partitions

Examples: PostgreSQL (single-node), Google Spanner, CockroachDB, Zookeeper

Use for: bank balances, inventory (don't oversell), booking systems, user auth

EVENTUAL CONSISTENCY

Replicas will converge to the same value eventually — but reads may be stale temporarily.

"Given enough time with no new writes, all replicas will agree."

Cost: temporary inconsistency that application must handle

Examples: Cassandra, DynamoDB, S3, DNS, social media feeds

Use for: social media likes/views, shopping carts (add item locally, sync later), DNS propagation, player scores in games, search index updates

CAP THEOREM APPLIED

You can only pick 2 of 3:

- Consistency + Partition Tolerance (CP): ZooKeeper, HBase — unavailable during network partition
- Availability + Partition Tolerance (AP): Cassandra, DynamoDB — returns stale data but stays up
- CA: traditional RDBMS — not partition-tolerant (single node)

PACELC: extends CAP — even without partitions, tradeoff between latency (L) and consistency (C)

PRACTICAL PATTERNS

Read-your-writes consistency: after a user writes, they see their own write (even if others see stale)

→ Route user's reads to the same replica they wrote to (using sticky sessions or user-hash routing)

Monotonic reads: user never sees older data than they've already seen

→ Route all reads for a user to same replica

WOW FACTOR

- Amazon's Dynamo paper (2007): foundational — chose AP over CP, introduced vector clocks
- "Conflict-free" data types (CRDTs): mathematical structures that merge concurrent edits without conflicts — Riak, Redis
- Cassandra's tunable consistency: set W (write quorum) + R (read quorum); if W+R > N = strong consistency

sd-64 · Tradeoffs

Synchronous vs Asynchronous Communication — when to use each

CLARIFY FIRST

One of the most impactful architectural decisions — affects latency, reliability, and coupling.

SYNCHRONOUS (REQUEST/RESPONSE)

Caller waits for response before continuing.

Examples: REST API calls, gRPC, database queries

Pros:

Simple mental model — call, get response, continue

Immediate feedback — errors surface instantly

Easy to debug and trace

Cons:

Caller blocked until response arrives

Caller inherits callee's downtime (tight coupling)

Cascade failures — slow service slows everything upstream

Timeout tuning is tricky

Use for: user-facing APIs (need result to render page), auth checks, read queries

ASYNCHRONOUS (EVENT/MESSAGE)

Caller publishes message and continues immediately. Consumer processes later.

Examples: Kafka, SQS, RabbitMQ, email systems

Pros:

Caller not blocked — better throughput

Loose coupling — services independent

Natural backpressure — queue absorbs traffic spikes

Consumer failure doesn't affect producer

Cons:

No immediate result — harder UX patterns ("we'll email you when done")

Message ordering and exactly-once delivery complexity

Harder to debug — distributed traces needed

Use for: sending emails/notifications, order processing, video transcoding, search index updates, analytics events

PRACTICAL PATTERNS

Async with callback: producer sends job, consumer calls webhook when done

Async with polling: producer returns job_id, client polls GET /jobs/{id}

Hybrid: synchronous for user response, async fan-out for side effects

COMMON MISTAKE

Everything synchronous → slow chain: user request triggers 6 sync service calls → 200ms becomes 1.2s

WOW FACTOR

- Amazon's order pipeline: payment sync (need to know immediately), fulfillment async (kick off after)
- The "two generals problem": proves 100% reliable synchronous communication over unreliable network is impossible
- Event-driven + CQRS: writes go async via events; reads from pre-built read models — massive read scalability

sd-65 · Tradeoffs

Concurrency vs Parallelism, and Load Balancing strategies

CLARIFY FIRST

Often confused in interviews — know the precise difference and practical implications.

CONCURRENCY vs PARALLELISM

Concurrency: dealing with multiple tasks at once (may not execute simultaneously)

→ Single CPU switching between tasks (interleaving)

→ Example: Node.js event loop — single thread handles thousands of connections via async I/O

Parallelism: actually executing multiple tasks simultaneously

→ Multiple CPUs/cores running at the same time

→ Example: Java thread pool processing requests on 8 CPU cores simultaneously

"Concurrency is about structure, parallelism is about execution" — Rob Pike

When each matters:

I/O-bound work (DB queries, network calls): concurrency wins → use async/event loop or goroutines

CPU-bound work (image processing, ML inference): parallelism wins → use thread pool or multiple processes

LOAD BALANCING STRATEGIES

Round Robin: distribute requests sequentially across servers

→ Simple, works when requests are roughly equal cost

Weighted Round Robin: servers with more capacity get proportionally more requests

→ Good for heterogeneous fleet

Least Connections: route to server with fewest active connections

→ Best for variable-length requests (some requests take 100ms, others 5s)

IP Hash / Sticky Sessions: same client always hits same server

→ Required for stateful services; breaks if server dies

Least Response Time: route to fastest responding server

→ Most dynamic, adapts to real performance but needs measurement overhead

LAYER 4 vs LAYER 7

L4 (Transport): routes by IP + port — fast, no content inspection (HAProxy, AWS NLB)

L7 (Application): routes by URL, headers, cookies — smarter, enables A/B testing (Nginx, AWS ALB)

HEALTH CHECKS

Active: load balancer sends /health every 5s, removes unresponsive servers

Passive: monitor real traffic error rates, remove servers exceeding threshold

WOW FACTOR

- Go's goroutines: 2KB stack vs Java threads 1MB — can run 1M concurrent goroutines on one machine
- "C10K problem" (1999): how to handle 10K concurrent connections — solved by async I/O (Node.js, Nginx)
- Consistent hashing for load balancing: adding/removing servers only remaps ~1/N of keys